

Luigi SEGALA M1-IA

Rapport de Projet

Classification d'intentions client

pour un système de support automatisé

Dataset : Customer Support on Twitter (Kaggle)

9 notebooks • 6 architectures • Déploiement FastAPI

I. Introduction et choix du dataset

Dans le cadre de ce projet, l'objectif était de concevoir, entraîner, comparer et déployer un système de classification d'intentions basé sur le Deep Learning. Le but final est de permettre à une entreprise de router automatiquement ses messages clients vers la bonne catégorie d'intention, afin d'accélérer le traitement du support.

Choix du dataset

J'ai choisi l'option alternative : Customer Support on Twitter (Kaggle), qui contient environ 3 millions de tweets. Ce choix se justifie par plusieurs raisons.

Premièrement, ce dataset est beaucoup plus réaliste et représentatif d'un système de support réel. Les tweets sont bruités, multilingues, avec des abréviations, des emojis et une grande variété d'expressions. C'est donc bien plus intéressant d'un point de vue technique que le dataset Bitext qui est déjà propre et équilibré.

Deuxièmement, ce dataset m'intéresse pour des raisons personnelles : comprendre et classifier automatiquement des messages courts en langage naturel est quelque chose de très utile dans de nombreux projets. Cela me donnera une méthodologie et une façon de procéder, ainsi qu'une compréhension des modèles plus forte, pour m'aider dans mes projets.

II. Préparation des données

2.1) Chargement et diagnostic

Le dataset twcs.csv contient environ 3 millions de tweets, dont on isole les tweets clients (inbound=True), soit 1 511 278 tweets. Avant toute modification, j'ai réalisé un diagnostic complet pour vérifier la présence de NaN, doublons et tweets trop courts. Sur ce dataset, les NaN et doublons étaient inexistantes ; j'ai donc uniquement supprimé les tweets de moins de 3 caractères.

2.2) Nettoyage textuel orienté intention

La fonction de nettoyage appliquée est `clean_tweet_intent`. Elle supprime les @mentions (bruit de marque), les URLs, les hashtags, les chiffres, et la ponctuation, à l'exception de trois marqueurs essentiels conservés volontairement :

Marqueur	Signal d'intention
?	Demande d'information, cela distingue la classe question des autres
!	Intensité émotionnelle, cela renforce complaint ou acknowledgment
' (apostrophe)	Préserve can't, don't, won't, qui sont des marqueurs de négation critiques

Les stopwords ont été conservés pour l'embedding et filtrés uniquement à l'affichage des mots-clés lors de l'exploration KMeans. Les mentions comme @AppleSupport ou @AmazonHelp ont été supprimées car elles poussaient KMeans à créer des clusters par marque et non par intention.

2.3) Exploration KMeans et labellisation

Pour découvrir les groupes naturels dans les données, j'ai encodé 200 000 tweets avec `cardiffnlp/twitter-roberta-base` (pré-entraîné sur 58 millions de tweets) puis appliqué KMeans pour K allant de 4 à 14. J'avais d'abord testé `all-MiniLM-L6-v2`, modèle généraliste, mais les clusters obtenus étaient peu lisibles, car les tweets Twitter sont trop bruités et spécifiques pour un modèle

entraîné sur du texte générique. Passer à un modèle pré-entraîné spécifiquement sur des tweets a produit des clusters nettement plus cohérents.

L'exploration a révélé quatre clusters géométriquement séparables quel que soit K : acknowledgment, non_english, question, et un large cluster général. À partir de K=8, deux clusters supplémentaires émergent partiellement : problem_report et complaint. Cependant, KMeans échoue à découvrir des classes d'intention claires, car les embeddings capturent la similarité sémantique du contenu (iPhone, colis) qui efface les frontières d'intention. J'ai donc opté pour une labellisation par règles lexicales.

2.4) Classification des 8 intentions

Classe	Fonction communicative	Exemple
non_english	Tweets non-anglais (filtrage)	Hola @AmazonHelp me pueden...
acknowledgment	Confirmer, remercier brièvement	Thanks, sent!
question	Demander une information	When will my order arrive?
problem_report	Décrire un dysfonctionnement	App keeps crashing since update
complaint	Exprimer frustration	This is absolutely unacceptable!!
follow_up	Relancer une interaction	Still waiting, any update?
help_request	Demander une action	Please fix this, I need help urgently
general	Catch-all	Tout le reste ambigu

2.5) Distribution, équilibrage et split

La distribution obtenue sur 1 511 278 tweets montre que general représente 70,7% du dataset, suivi de question à 11,8%. Les classes minoritaires comme follow_up (1,3%) ou help_request (2,0%) sont bien présentes. Pour éviter que le modèle ignore ces classes, j'ai plafonné chaque classe à 100 000 exemples, aboutissant à un dataset équilibré de 465 000 tweets environ.

Le split a été réalisé de façon stratifiée : 70% pour l'entraînement, 15% pour la validation et 15% pour le test. Le paramètre stratify garantit que chaque split respecte les proportions de classes.

Le vocabulaire a été construit uniquement à partir du train (15 000 tokens les plus fréquents) pour éviter tout data leakage. Deux tokens spéciaux ont été ajoutés : <PAD> (id=0) pour le padding et <UNK> (id=1) pour les tokens hors-vocabulaire.

III. Description des architectures testées

Six architectures ont été implémentées et comparées. Les quatre premières et CNN+BiLSTM partagent le même pipeline de données (client_data.pkl, vocab.pkl) et la même boucle d'entraînement. DistilBERT utilise un pipeline séparé (client_text_data.pkl) car il dispose de son propre tokenizer WordPiece.

3.1) Architecture 1 : RNN

Le RNN traite la séquence token par token. À chaque pas t , il met à jour un état caché qui résume tout ce qui a été lu selon la formule : $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$.

La limite principale est le vanishing gradient : lors de la rétropropagation, les gradients sont multipliés à travers tous les pas de temps et tendent vers 0 sur des séquences longues. Pour

contourner ce problème, j'ai utilisé `output.mean(dim=1)` (moyenne sur tous les états cachés) plutôt que `hidden[-1]`, afin de préserver l'information du début de séquence.

Paramètres : `EMBED_DIM=128`, `HIDDEN_DIM=256`, `DROPOUT=0.3`, `LR=1e-3`, `BATCH_SIZE=64`, `N_EPOCHS=10`. Un gradient clipping (`max_norm=1.0`) est appliqué pour éviter l'explosion de gradient.

3.2) Architecture 2 : LSTM et BiLSTM

Le LSTM résout le vanishing gradient grâce à un cell state `c_t` (mémoire long terme) et trois portes : forget gate (quoi oublier), input gate (quoi ajouter), output gate (quoi exposer). La mise à jour par addition est la clé : le gradient remonte sans s'atténuer.

Le BiLSTM ajoute une deuxième couche qui traite la séquence en sens inverse. Les deux derniers états cachés sont concaténés : `torch.cat([hidden[-2], hidden[-1]], dim=1)`. Le modèle dispose ainsi du contexte passé et futur pour chaque token, ce qui est particulièrement utile pour les tweets où la fin de phrase conditionne souvent l'intention.

Paramètres : `EMBED_DIM=128`, `HIDDEN_DIM=256`, `NUM_LAYERS=2`, `DROPOUT=0.3`, `LR=5e-4`, `BATCH_SIZE=64`, `N_EPOCHS=15`.

3.3) Architecture 3 : Transformer from scratch

Le Transformer utilise un mécanisme d'attention multi-têtes (Multi-Head Self-Attention) pour pondérer chaque token en fonction de tous les autres. Contrairement au RNN, il n'a pas d'état caché séquentiel et traite toute la séquence en parallèle.

Chaque TransformerBlock suit le schéma : `LayerNorm` → `MultiHeadSelfAttention` → `Dropout` → résidu, puis `LayerNorm` → `FFN` → `Dropout` → résidu. Les connexions résiduelles permettent au gradient de circuler sans s'atténuer. Un masque de padding est appliqué pour ignorer les tokens `<PAD>`. La représentation finale est obtenue par moyenne masquée sur les tokens non-PAD.

Paramètres : `EMBED_DIM=128`, `NUM_HEADS=4`, `FF_DIM=256`, `NUM_BLOCKS=2`, `DROPOUT=0.1`, `LR=1e-3`, `N_EPOCHS=10`.

3.4) Architecture 4 : DistilBERT fine-tuné

Contrairement aux architectures précédentes entraînées from scratch sur 465 000 tweets, DistilBERT est pré-entraîné sur 11 milliards de tokens (Wikipedia + BooksCorpus). C'est une version distillée de BERT : 6 couches au lieu de 12, 66M paramètres, qui préserve 97% des capacités avec 40% moins de paramètres.

Son tokenizer `WordPiece` décompose les mots rares en sous-mots (ex. « cancellation » → `[cancel, ##ation]`), ce qui élimine les `<UNK>` pour les noms de marques ou abréviations Twitter, et c'est donc un avantage direct sur notre vocab custom de 15 000 tokens. Le premier token de la séquence résume l'ensemble du texte et sert d'entrée au classifieur.

Paramètres : `MAX_LEN=128`, `BATCH_SIZE=32`, `LR=2e-5` (plage standard BERT), `N_EPOCHS=3`, AdamW avec `weight_decay=0.01`.

3.5) Architecture 5 : CNN + BiLSTM (hybride)

Cette architecture hybride combine deux étapes complémentaires : un CNN 1D qui détecte les patterns locaux via des filtres glissants sur 3 tokens consécutifs ("app not working", "please help me", "still no response"), suivi d'un BiLSTM qui modélise les relations entre ces patterns dans la phrase.

Cette combinaison est particulièrement adaptée à la classification d'intention sur tweets, car les intentions se manifestent souvent par des expressions locales courtes. Le CNN les détecte mieux qu'un LSTM seul qui traite les tokens un par un. Le BiLSTM prend ensuite ces signaux comme entrée et modélise leurs relations dans la phrase. De plus, le CNN réduit la longueur de séquence (64 → 62 pour `kernel=3`), ce qui atténue le vanishing gradient du BiLSTM.

Paramètres : `EMBED_DIM=128`, `NUM_FILTERS=256`, `KERNEL_SIZE=3`, `HIDDEN_DIM=256`, `DROPOUT=0.3`, `LR=5e-4`, `BATCH_SIZE=128`, scheduler `ReduceLROnPlateau`.

IV. Protocole expérimental

Pour garantir une comparaison équitable, tous les modèles from scratch partagent le même pipeline de données (client_data.pkl), la même boucle d'entraînement et la même stratégie d'évaluation.

4.1) Environnement d'entraînement

Les modèles from scratch (RNN, LSTM, BiLSTM, Transformer, CNN+BiLSTM) ont été entraînés avec les configurations décrites ci-dessus. DistilBERT a été fine-tuné avec le modèle de base distilbert-base-uncased sur GPU P100 (VRAM 16 GB, BATCH_SIZE=32). Les embeddings RoBERTa pour le clustering ont été calculés avec BATCH_SIZE=256 sur le même GPU.

4.2) Boucle d'entraînement

Pour chaque époque : entraînement avec model.train() (passe forward + backward sur le train set), évaluation avec model.eval() et torch.no_grad() sur le val set, sauvegarde du meilleur état si val_acc s'améliore. À la fin, chargement du meilleur état et évaluation finale sur le test set (touché une seule fois).

4.3) Métriques d'évaluation

Pour chaque modèle, les métriques suivantes ont été calculées : accuracy globale, precision/recall/F1 par classe via classification_report, matrice de confusion (heatmap seaborn), courbes train/val loss et accuracy.

Cependant, latence d'inférence (200 répétitions, 20 warmup) est calculé que pour le BiLSTM car c'est ce modèle qu'on va choisir pour optimiser et réguler (voir section VI).

V. Résultats commentés

5.1) Tableau comparatif

Modèle	Test Accuracy	Best Val Acc	Epochs
DistilBERT (fine-tuné)	99.50%	99.53%	3
CNN + BiLSTM (hybride)	97.92%	97.86%	15
BiLSTM	97.87%	97.76%	15
LSTM	97.83%	97.81%	15
Transformer from scratch	96.71%	96.73%	10
RNN	95.66%	95.65%	10

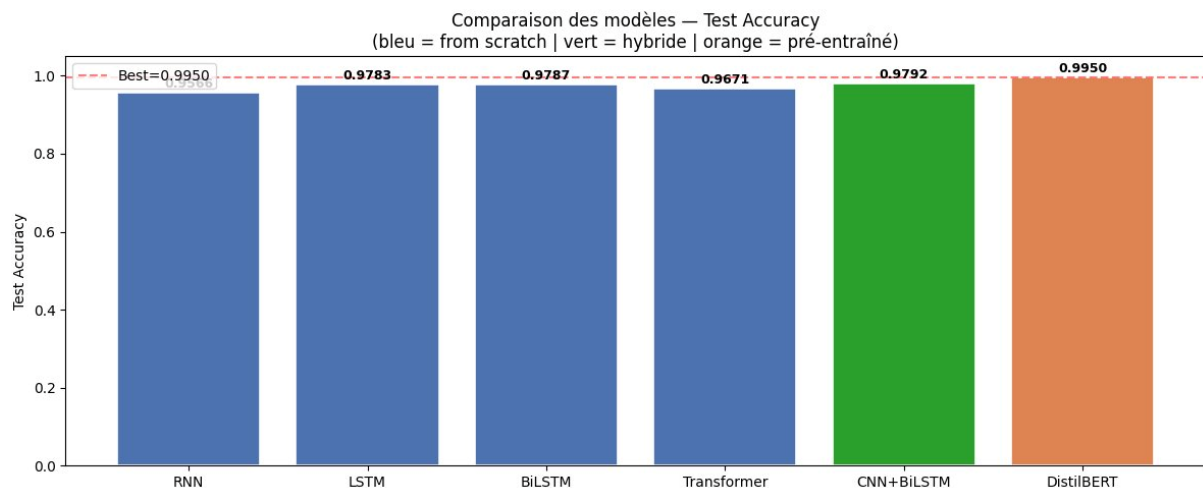


Figure 1 : Comparaison des modèles - Test Accuracy

5.2) Analyse des résultats

DistilBERT se détache clairement avec environ 2% d'écart sur les modèles from scratch. Cet écart reflète la valeur du pré-entraînement sur 11 milliards de tokens : ses représentations contextuelles capturent des nuances que les embeddings custom (vocabulaire de 15 000 tokens, entraînés from scratch) ne peuvent pas atteindre. Il converge de plus en seulement 3 époques, là où les modèles from scratch en nécessitent 10 à 15.

Les modèles from scratch se regroupent autour de 97-98%, car ils apprennent les patterns lexicaux directs des règles. Le CNN+BiLSTM (97.92%) dépasse légèrement le BiLSTM seul (97.87%) grâce à la détection de trigrammes par le CNN. Le Transformer from scratch (96.71%) est en dessous des architectures récurrentes, ce qui s'explique par l'absence de pré-entraînement : sur des tweets courts (moyenne 17 tokens), ses avantages (attention globale, parallélisme) ne compensent pas encore le manque de données d'entraînement.

Le RNN (95.66%) confirme sa limite : le vanishing gradient l'empêche de capturer certaines dépendances même sur des séquences courtes, malgré le mean pooling.

5.3) Courbes d'apprentissage

Les courbes train/val loss des modèles from scratch montrent une convergence stable sans surapprentissage notable, grâce au dropout (0.3) et à l'équilibrage du dataset. L'écart entre train_acc et val_acc reste inférieur à 1% pour le BiLSTM et le CNN+BiLSTM. Pour DistilBERT, la val_loss décroît rapidement et se stabilise dès l'époque 2, signe que le modèle est déjà bien adapté à la tâche.

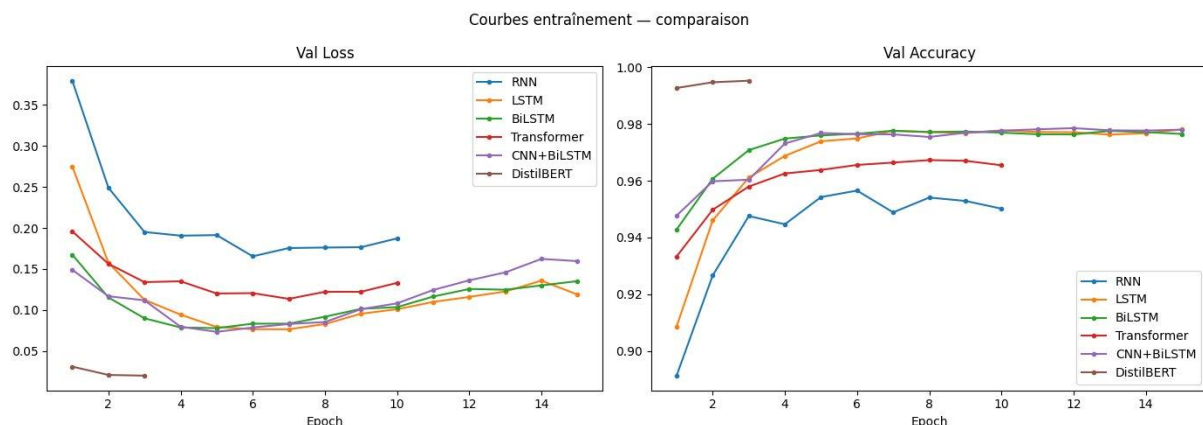


Figure 2 : Courbes d'entraînement (val loss et val accuracy) - comparaison des 6 architectures

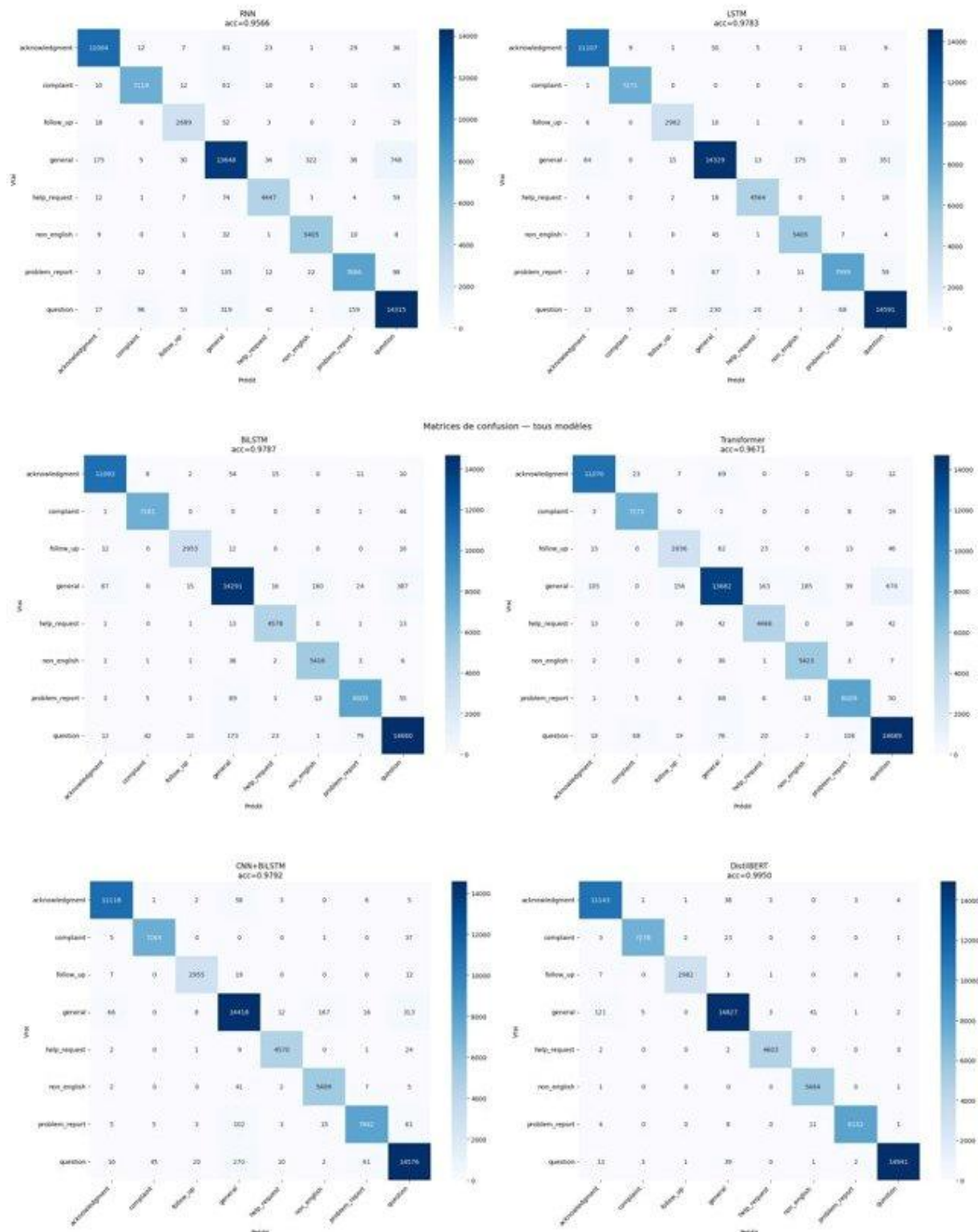


Figure 3 : Matrices de confusion - comparaison des 6 modèles

VI. Optimisation et régularisation du BiLSTM

J'ai gardé le BiLSTM pour l'optimisation et le déploiement, et non DistilBERT, car le BiLSTM est 15× plus léger (~17 Mo vs 268 Mo). De plus, DistilBERT à 99.50% est déjà à saturation sur ce dataset et les techniques d'optimisation classiques n'y apporteraient pas de gain mesurable.

6.1) Comparaison d'optimiseurs (SGD, Adam, AdamW)

Avant de choisir le scheduler et la régularisation, trois optimiseurs ont été comparés sur le BiLSTM baseline (mêmes hyperparamètres : LR=5e-4, dropout=0.3), entraînés 5 époques chacun :

Optimiseur	Best Val Acc (5 époques)
SGD (momentum=0.9)	0.7419
Adam	0.9764
AdamW	0.9763

SGD+momentum reste très en retard (74.19%) après 5 époques : sa convergence est beaucoup plus lente car le LR=5e-4 (calibré pour Adam) est trop faible pour SGD, qui n'a pas de taux d'apprentissage adaptatif par paramètre. Adam et AdamW convergent quasiment à la même vitesse et atteignent ~97.6% en seulement 5 époques. AdamW est très légèrement en dessous d'Adam ici (0.9763 vs 0.9764), l'effet du weight decay découplé se manifeste surtout sur un entraînement plus long. Adam est donc conservé pour la suite de l'optimisation (sections 6.3 à 6.7), le weight decay étant ajouté séparément lors de la régularisation.

6.2) Schedulers de learning rate

Deux schedulers ont été comparés. ReduceLRonPlateau divise le LR par 0.5 si la val_loss stagne pendant 2 époques (profil en escalier). CosineAnnealingLR applique une décroissance cosinus sur T_max=15 époques (décroissance continue progressive). Le scheduler CosineAnnealingLR produit un meilleur résultat (val_acc 0.9779 vs 0.9776) grâce à une descente de gradient plus fluide.

Une petite précision, le LinearWarmup n'a pas été utilisé car il est spécifique aux modèles pré-entraînés (BERT). Le BiLSTM est entraîné from scratch, un scheduler classique suffit donc ici.

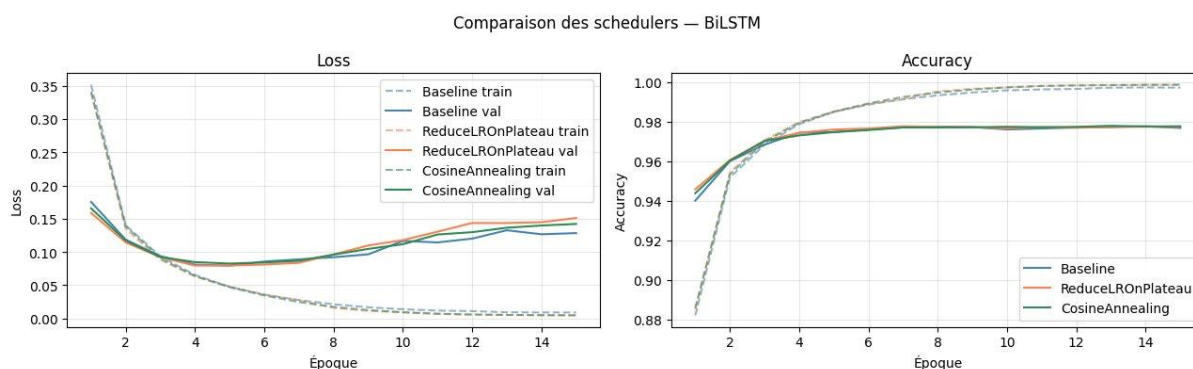


Figure 4 : Comparaison des schedulers (ReduceLRonPlateau vs CosineAnnealingLR) - BiLSTM

6.3) Early Stopping

L'early stopping arrête l'entraînement si val_acc ne s'améliore pas pendant patience=4 époques consécutives et restaure les poids du meilleur état. C'est une différence importante avec le scheduler : le scheduler adapte le LR pour continuer à apprendre malgré un plateau, tandis que l'early stopping arrête l'entraînement pour éviter l'overfitting.

6.4) Régularisation

Trois techniques de régularisation ont été étudiées. Le Dropout (varié de 0.1 à 0.5) force le modèle à ne pas dépendre d'un neurone unique, favorisant la généralisation. Le Weight Decay (1e-4 dans Adam) pénalise les poids trop grands, évitant l'overfitting sur le vocabulaire. L'ajout d'une deuxième couche LSTM (num_layers=2) permet à la couche 1 d'apprendre les patterns lexicaux et à la couche 2 les relations entre patterns.

6.5) LR différencié et configuration finale

Un LR différencié par groupe a été appliqué : l'embedding reçoit un LR de $LR \times 0.5$ (converge plus vite), le LSTM LR standard, et la tête de classification $LR \times 2$ (spécialisation plus rapide sur la tâche).

Configuration finale retenue : dropout=0.3, num_layers=2, CosineAnnealingLR (T_max=20), patience=5, num_epochs=20.

6.6) Recherche d'hyperparamètres

Une Grid Search sur 9 combinaisons de (LR, dropout) a été réalisée, évaluée sur 3 époques. $LR=1e-3$ domine sur 3 époques (convergence plus rapide). $LR=1e-4$ est trop faible, le modèle est sous-entraîné.

Une Random Search complémentaire (9 tirages) a exploré l'espace continu avec LR log-uniforme entre $1e-4$ et $1e-3$ et dropout uniforme entre 0.1 et 0.5. Le log-uniforme pour LR est justifié par le fait que les ordres de grandeur comptent plus que les valeurs absolues.

Les résultats sont visualisés via une heatmap (Grid Search) et un scatter plot en échelle log (Random Search), permettant de lire visuellement les zones optimales de l'espace d'hyperparamètres.

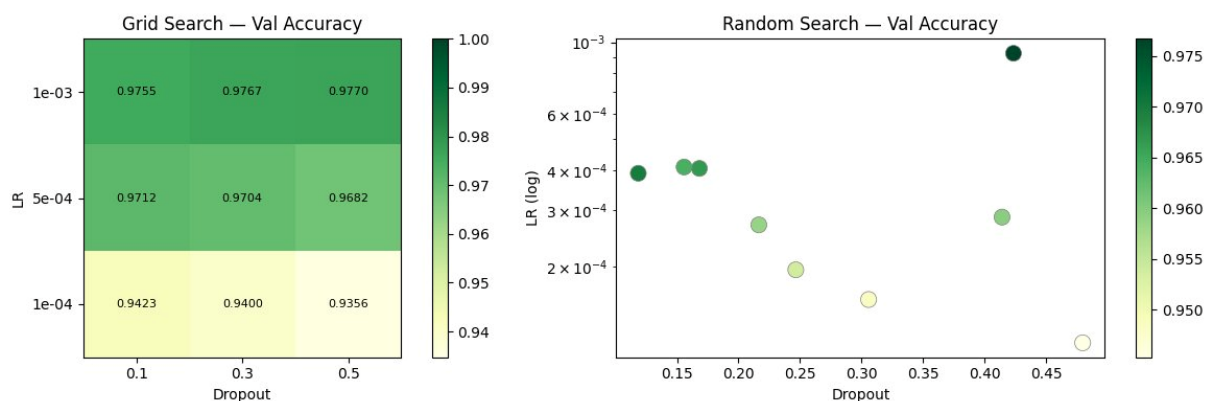


Figure 5 : Grid Search et Random Search - Val Accuracy

6.7) Résultats de l'optimisation

Configuration	Best Val Acc
Baseline (LR fixe)	0.9777
Sans régularisation	0.9779
ReduceLRonPlateau	0.9776
CosineAnnealingLR	0.9779
Early Stopping	0.9785
Dropout + Weight Decay	0.9786
Entraînement complet assemblé	0.9806
TEST SET	0.9808

Gain final : 97.87% (BiLSTM de base) $\rightarrow$ 98.08% (+0.21%).

VII. Analyse d'erreurs

7.1) Classes les plus difficiles

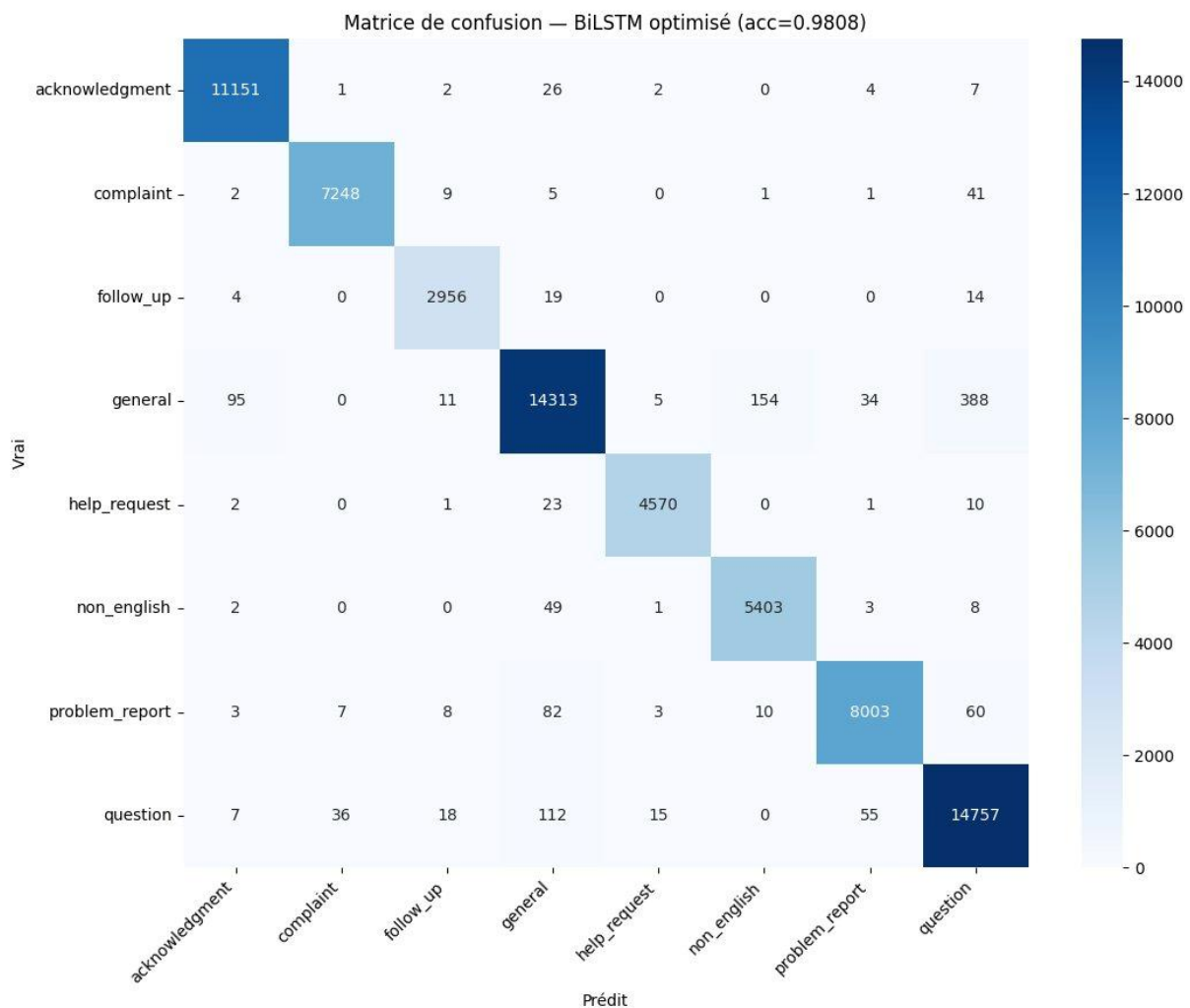


Figure 6 : Matrice de confusion - BiLSTM optimisé (acc=0.9808)

L'analyse par classe du BiLSTM optimisé sur le test set révèle deux classes particulièrement difficiles.

La classe general (recall 0.95) présente 687 tweets mal classés, principalement confondus avec question (388 cas) et non_english (154 cas). C'est normal : general est le catch-all par définition le plus ambigu.

La classe question (precision 0.97) voit 55 tweets problem_report et 112 tweets general prédits à tort comme question, car contenant un « ? » sans être de vraies demandes d'information. La classe complaint est aussi confondue avec question dans 41 cas : ce sont des plaintes formulées sous forme interrogative (Why is your service so terrible?).

7.2) Exemples concrets sur l'API déployée

Cinq cas ont été testés directement sur l'API en production.

Cas 1 : Cas ambigu classé general : « @AppleSupport fix your health app. I just went for a 1 hour 20 minutes jog. » → general (0.79), problem_report en 2e position (0.18). La mention est supprimée par le preprocessing et « fix your app » n'active aucun mot-clé exact. La confiance faible (0.79) témoigne que le modèle hésite, ce qui est cohérent.

Cas 2 : Question claire : « @AmazonHelp Where can I see ongoing Amazon Pay offers? » → question (0.9985). La mention est supprimée par le preprocessing. "Where" + "?" activent clairement la règle question. Confiance quasi-certaine (0.9985), cas idéal où forme et intention sont parfaitement alignées.

Cas 3 : Sarcasme, limite du modèle : « @comcastcares I'm paying \$80 a month instead of \$50... Thanks! » → general (0.9959). Sémantiquement c'est une plainte évidente, mais aucun mot-clé

explicite de complaint n'est présent. Le modèle reproduit fidèlement le label généré par les règles lors de l'entraînement. C'est la limite fondamentale : le modèle ne peut pas apprendre ce que les règles n'ont pas su capturer.

Cas 4 : Tweet agent (non-client) : « Can you send your full name, postcode & address and email please? Thanks » → general (0.9827). « Can you send » ne correspond à aucune règle `help_request`. Classifié general correctement, le modèle n'est pas trompé par la structure interrogative.

Cas 5 : Généralisation au-delà des règles : « @AppleSupport The article doesn't say anything about locking or pulling down cover sheet. Its probably a bug then. » → `problem_report` (1.0). Le mot "bug" suffit à déclencher une confiance maximale. C'est un exemple de généralisation réussie : le contexte ("doesn't say anything", "probably a bug") est ambigu mais le modèle l'associe correctement à un problème technique grâce aux patterns appris sur l'ensemble d'entraînement.

7.3) Limite fondamentale

Il y a une nuance importante à garder en tête : une accuracy élevée ne signifie pas que le modèle est fiable à 100%. L'accuracy mesurée (98.08%) reflète la capacité du modèle à reproduire les labels générés par les règles, pas sa capacité à comprendre l'intention réelle d'un tweet. Un tweet sarcastique sera classé general avec 99% de confiance, ce qui est « correct » vis-à-vis des labels d'entraînement, mais faux vis-à-vis de l'intention humaine réelle. La métrique d'accuracy est donc à interpréter dans le contexte de la qualité des labels.

VIII. Déploiement

8.1) Exports du modèle

Format	Taille	Latence (1 tweet, CPU)	Accuracy
PyTorch (base)	~16.37 Mo	~3.03 ms	98%
TorchScript (tracé)	~16.39 Mo	~2.94 ms	98%
Quantisé int8	~16.37 Mo	~3.48 ms	98%

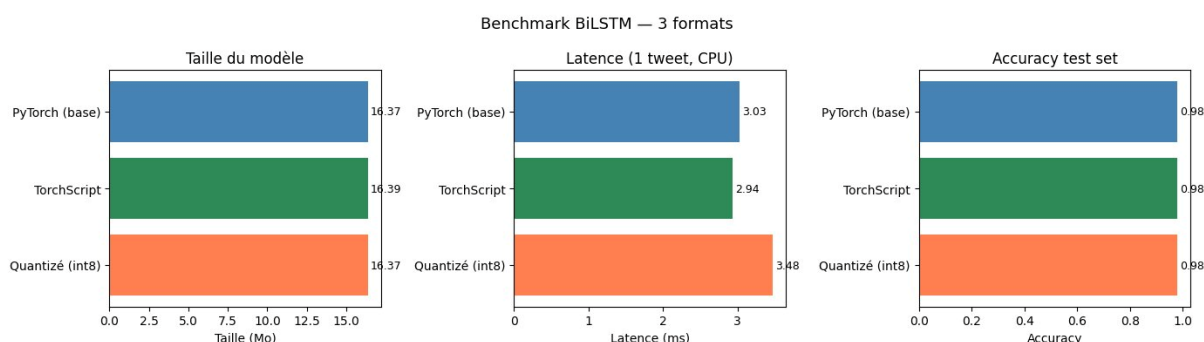


Figure 7 : Benchmark BiLSTM - 3 formats d'export (taille, latence, accuracy)

TorchScript a été généré via `torch.jit.trace` (architecture statique, sans branches conditionnelles dans `forward`). L'avantage est que le fichier `.pt` peut être chargé et exécuté en C++ sans Python installé.

ONNX (Open Neural Network Exchange) est un format universel compatible avec ONNX Runtime, TensorRT (NVIDIA), OpenVINO (Intel) et CoreML (Apple). L'export utilise `dynamic_axes` pour permettre des `batch_size` et `seq_len` variables.

La quantization dynamique (int8) réduit la précision des poids de float32 à int8 sur les couches Linear. En théorie ça donne un modèle plus léger et plus rapide, mais ici ça n'a presque rien changé. La raison est sûrement que seule la tête de classification est une couche Linear dans

notre BiLSTM, le reste (embedding et LSTM) n'est pas touché. En conclusion, on a la même taille, et même légèrement plus lent à cause de l'overhead de dé-quantization.

8.2) Pipeline d'inférence

Le pipeline d'inférence est strictement identique au preprocessing d'entraînement (clean_tweet → tokenize → pad_sequence → BiLSTM forward → softmax → top-k). Cette cohérence est critique : un preprocessing différent en production dégraderait les performances même si le modèle est parfait.

8.3) API FastAPI

L'API expose deux endpoints. POST /predict prend un texte brut et renvoie l'intention prédite, le score de confiance et le top-3 des intentions. GET /health renvoie le statut de l'API et le nom du modèle chargé.

La documentation Swagger est accessible automatiquement à /docs. L'API a été déployée et testée sur <http://51.15.234.209:8000/predict> avec des exemples réels (voir section analyse d'erreurs).

IX. Conclusion et pistes d'amélioration

9.1) Bilan

Ce projet est très intéressant et m'a permis de comprendre les enjeux de la mise en place d'un modèle de Deep Learning. De la préparation des données, au choix de l'architecture en passant par l'optimisation et le déploiement. C'était passionnant et exigeant car il y a une envie d'avoir une architecture parfaite, mais cela est compliqué avec peu de moyens humains et de temps.

Mais c'est en choisissant de me confronter à la difficulté avec le dataset Twitter que j'ai pu me rendre compte de tout cela.

Mon approche par supervision faible a permis de produire 465 000 paires (tweet, intention) exploitables sans annotation manuelle. Le BiLSTM optimisé (98.08%) a été retenu pour le déploiement pour ses performances opérationnelles : 16 Mo, 3 ms de latence, sans dépendance HuggingFace. DistilBERT reste le modèle le plus précis (99.50%) mais est inadapté à un déploiement temps réel à fort volume.

Ce projet peut être amélioré en enrichissant les labeling functions avec un outil de sentiment comme VADER pour mieux capturer les plaintes sarcastiques, ou en explorant une classification zero-shot pour s'affranchir des labels déterministes.

9.2) Pistes d'amélioration

Première piste : enrichir la supervision faible en combinant plusieurs signaux de labellisation comme les règles lexicales actuelles, un modèle de sentiment comme VADER pour détecter les plaintes implicites, et des patterns syntaxiques pour le sarcasme. C'est une démarche naturelle et légère qui ne nécessite pas de réentraîner quoi que ce soit.

Deuxième piste : la classification zero-shot ne nécessite aucune donnée labellisée et comprend le sarcasme nativement. Elle est cependant trop lente (~500 ms par tweet) pour un usage temps réel à l'échelle de millions de tweets.

Troisième piste : itérer sur les règles lexicales en analysant les exemples mal classés. Par exemple, enrichir follow_up avec « no news », « waiting since », « heard nothing » récupérerait des tweets qui tombaient dans general. Analyser les erreurs → identifier les mots manquants → enrichir les règles.

Quatrième piste : annoter manuellement les tweets qui sont dans general. C'est la solution la plus directe mais aussi la plus coûteuse en temps.

Le système actuel est solide pour les intentions explicites (97-99% d'accuracy sur les classes bien couvertes par les règles). Les limites apparaissent sur le sarcasme et les expressions implicites,

ces cas qui nécessitent une compréhension pragmatique que les labels keyword-based ne peuvent pas encoder.